

PART 1 · 讲课版

认知重建

Agent 到底是什么，别被概念忽悠

Harry 的 Agent 实战课 · 清华学长 harry | AI 产品手艺人

01 不是新物种

02 进化史

03 发动机不是车

04 成熟度分级



AI 生成

为什么第一件事，是 " 重建认知 " 而不是写代码

别急着学怎么搭。如果你连 " 自己做的到底是不是 Agent、该做到多 Agent" 都说不清，后面所有的技术选型都会跑偏。这一篇先把判断力立起来。

01

不是新物种

Agent 是自动化能力谱上 " 模型开始自主决策 " 的那一段，是连续谱不是开关

02

一张图看懂进化史

70 年里 " 让机器自主决策 " 几起几落，LLM 第一次补上了常识与语言这块拼图

03

大模型是发动机不是车

模型只是发动机，产品体验取决于它和 Prompt/ 工具 / 护栏配得好不好

04

产品成熟度分级 L0~L3

把模糊的 " 要不要做 Agent" 变成清晰的 " 该做到 L 几 " 的原创判断框架

开场

"我们这个，也是 Agent 产品。"

这句话我在大大小小的评审会上听过太多次。但仔细一问——**十次里有六次，其实只是 "调了个大模型 API 拼了个话术"。**

这不是挑刺，是产品经理最不该犯的错

如果你不能准确判断自己做的东西处在哪个技术层级，你就无法预估它的成本、风险和天花板。所以这一篇的第一件事——不是学怎么 "做" Agent，是先学会怎么 "识别" Agent。

Agent 不是新物种

从自动化脚本到自主智能体的连续谱——判断一个系统是不是 Agent，不看框架，看决策权在谁手里。

Agent 的本质：一个 " 感知—决策—行动 " 的闭环

抛开所有花哨的定义，Agent 的核心就是三件事反复循环：



关键三变量：这个循环 由谁驱动、能不能自主终止、遇到意外能不能自己调整 —— 决定了一个系统离 " 真正的 Agent" 有多远。

一条连续谱，而不是一个开关

把 " 自主性程度 " 当作横轴，从固定脚本到多智能体协作，是一条平滑上升的谱系：

位置	名称	决策由谁做	典型例子
1	固定脚本 / RPA	人（写死在代码里）	定时爬虫、Excel 宏
2	条件流程编排	人预设的 if-else 树	传统客服机器人、Dify workflow
3	单轮工具调用 LLM	LLM 决定调哪个工具，但只有一步	" 帮我查下天气 " 类插件
4	自主规划循环（ReAct）	LLM 自己规划多步、判断是否结束	AutoGPT 类、深度研究 Agent
5	多智能体协作	多个 LLM 角色协商、分工、互检	多角色写作 / 开发团队模拟

关键认知： Agent 不是 "5" 独有的称号，而是谱上 " 自主性明显跃升 " 的 3~5 段的统称。越往右能力越高，但不可控性、成本、调试难度也越高——不是越 Agent 越好，是够用就好。

三个最常见的误区

误区一

"调了 LLM API 就是 Agent"

只是把问题拼进 prompt 让模型直接答，中间没有 "要不要调工具、调哪个、要不要继续" 的决策分支——那是 LLM 应用，不是 Agent。

误区二

" workflow编排 = Agent"

Dify/Coze/n8n 本质是条件流程编排（位置 2），路径是人预先画好的。它能解决 80% 场景，但和 "模型自主规划路径" 是两回事。

误区三

" 必须多智能体才叫高级 "

多智能体的复杂度、调试成本、Token 消耗是单 Agent 的数倍。绝大多数业务，一个设计良好的单 Agent（位置 4）就足够。

产品经理的三个问题，定位你的产品

拿到一个 "Agent 产品 " 需求时，先别急着讨论技术方案，先问自己三个问题：

Q1

需要自己决定 " 要不要调工具、调几次 " 吗？

✗ 不需要 → 是流程自动化，别叫 Agent，用更便宜的方案

✓ 需要 → 至少 位置 3 起步

Q2

需要在多步之间自主判断 " 任务是否完成 " 吗？

✗ 不需要 → 停在位置 3，别过度设计

✓ 需要（模糊终点）→ 需要 位置 4 自主规划循环

Q3

任务天然需要多个专业角色分工、且单角色扛不住？

✗ 不是 → 先把单 Agent 做扎实

✓ 是 → 才考虑 位置 5 多智能体

CHAPTER 02

一张图看懂 Agent 进化史

70 年里 " 让机器自主决策 " 几起几落，每一次都被同一个问题卡住——直到 LLM 出现。

"Agent 是不是又一阵风？"——用历史回答

过去 70 年，"让机器自主做决策"已经经历过几轮兴衰。每一轮都被同一个问题卡住：

机器没有足够的 " 常识 " 和 " 语言理解能力 "，去处理开放世界的模糊性。

这一轮为什么不一样

LLM 第一次同时解决了 " 常识 " 和 " 语言理解 " 这两个问题——这才是这轮浪潮和前几次本质上的区别。所以它不是风口，是补上了拼图的最后一块。

70 年，六个阶段



LLM 的三合一：为什么是它终结了几十年的僵局

前四个阶段各自解决了 " 逻辑 / 知识 / 试错 / 感知 " 中的一项，但都缺 " 自然语言理解 + 常识推理 " 这块粘合剂。LLM 第一次把三者叠在一起：

1 世界常识

海量文本预训练带来的常识，不用再手写规则

2 语言理解与生成

能和人无缝自然交互，而不是填表式命令

3 推理与规划

一定程度拆解多步任务，具备 " 想下一步 " 的能力

三者叠加 = " 自主决策的智能体 " 第一次具备了通用性，不再局限于围棋那种封闭环境。

范式转移的规律：历史没有被推翻，是被拼接

每一代范式的死因，都是 " 底层能力 " 跟不上 " 应用场景的开放程度 "，而不是 " 这个方向本身错了 "。

符号主义的遗产

" 领域规则库 " 思想，今天活在 " 给 Agent 设计护栏和工具白名单 " 里

行为主义的遗产

" 奖励设计 " 思想，直接决定你的 Agent 训练 / 评估该怎么打分

给 PM 的实操启示：不要因为某个技术路线 " 过时 " 就抛弃它的方法论。判断一个新技术能不能用，看它解决了历史上哪个具体瓶颈，而不是只看它 " 新不新 "。

大模型是发动机，不是车

选模型不是看跑分榜马力，是看发动机和整车——Prompt、工具链、护栏——配得好不好。

模型只是发动机，产品体验来自 " 整车装配 "



产品经理必须建立的三层 " 发动机认知 "

1

第一层

本质是 " 续写 " 不是 " 理解 "

模型核心任务是 " 给定前文预测下一个词 "。它没有事实核查机制——这正是幻觉的根源，也是它对 " 离题 " 极其敏感的原因。

2

第二层

Prompt 工程的三个层次

L1 指令 / L2 示例 (Few-shot) / L3 结构化约束。只要 Agent 要用代码解析模型输出去决定动作，就必须上 L3，否则 " 自主决策 " 会变成 " 随机崩溃 "。

3

第三层

按场景选模型 + 分层调度

别看跑分榜排名，按场景（长文本 / 强推理 / 低成本）选。更别用一个模型扛全部——小模型做路由分类，大模型做真正的推理。

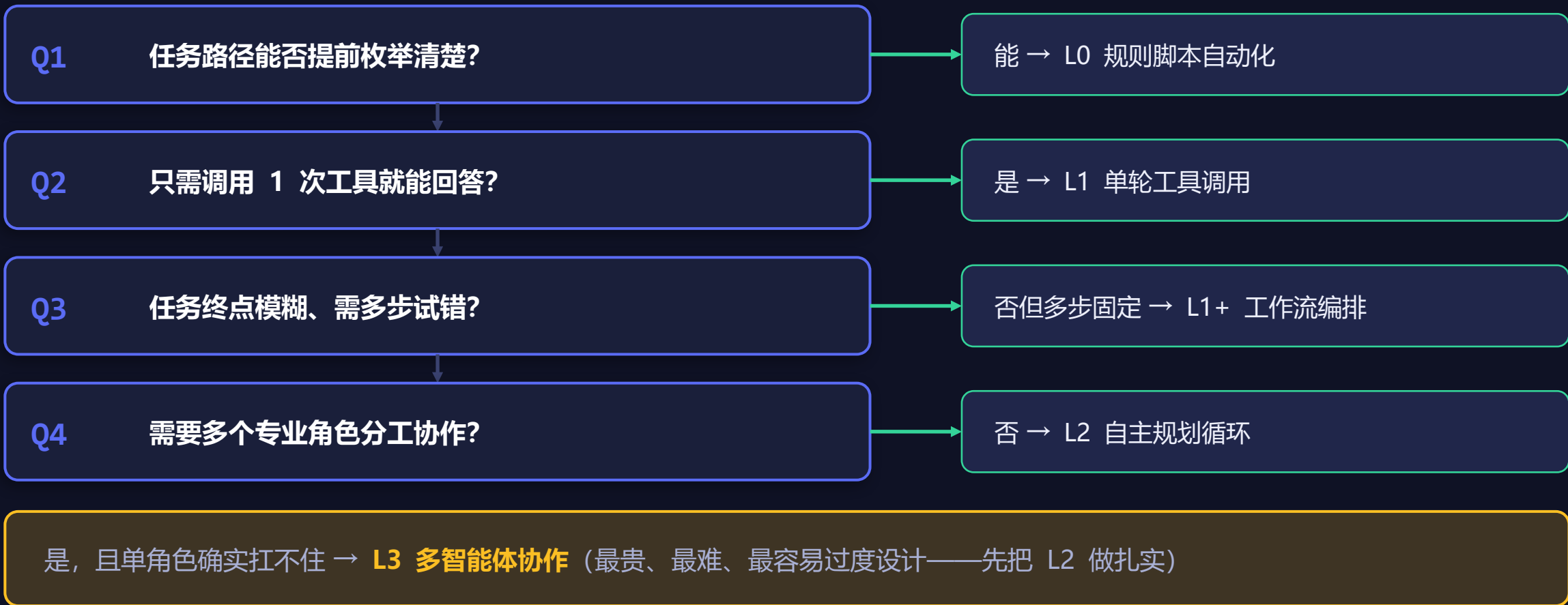
三大局限，直接倒逼你的 Agent 设计

局限	具体表现	对 Agent 设计的直接倒逼
幻觉	会自信地编造不存在的事实 / API / 数据	→ 设计 " 结果校验环节 "—— 让工具返回真实数据，关键结论要有可追溯来源
上下文窗口有限	任务变长后，前面的信息会被挤出去或稀释	→ 设计记忆架构（短期上下文 + 长期检索），别假设模型 " 记得住 "
成本与延迟	强模型又贵又慢，不可能每步都用最强	→ 分层调度：路由用小模型，推理用大模型，可复用结果做缓存

Agent 产品成熟度分级 L0~L3

"要不要做 Agent" 是假问题。真正该问的是——我们的产品需要做到哪一级的自主性？

四个问题，定位你该做到 L 几



L0–L3 分级对照速查表

	L0 规则脚本	L1 单轮工具	L2 自主循环	L3 多智能体
决策者	人 / 规则	模型（单步）	模型（多步自主）	多个模型角色
路径确定性	完全确定	基本确定	边做边定	不确定 + 分角色
典型开发	纯代码	Function Calling	ReAct / Plan-Solve	Multi-Agent 框架
调试难度	低	低	中高	高
Token / 成本	几乎无	低	中高（多轮）	高（多 Agent）
何时引入	需求确定不变	MVP 默认起步	L1 不够用再升	L2 扛不住再升

升到下一级之前，先过这份自问清单



我是否已经 " 验证 " 了当前级别真的不够用——还是只是主观觉得 " 应该更高级 " ？



升一级的边际收益（体验提升）和边际成本（Token / 延迟 / 可控性下降），我算过账吗？



如果要升到 L2，我给自主循环设计的护栏（最大步数、结果校验）是什么？



如果要升到 L3，我能不能明确说出 " 哪个具体角色分工是单 Agent 顾不过来的 " ？说不出就别升。

Part 1 建立的四个核心认知

01

Agent 是连续谱，不是开关

看决策权在谁手里，够用就好

02

每代范式兴衰都有清晰逻辑

LLM 补上了常识与语言这块拼图

03

大模型的能力边界决定设计

发动机不是车，局限倒逼护栏与调度

04

一套可直接用的分级框架

L0~L3，把 "要不要" 变成 "该做到几"

NEXT · 下一篇

Part 2 · 亲手造轮子

从 Prompt 到框架——手写三大经典范式、判断该不该用低代码平台、主流框架产品化评测、从零写一个 <300 行的 Agent 框架。认知立住了，接下来就该动手了。

配套学起来

交互式学习机（闯关学）

harryjzhang69-web.github.io/harry-agent-course/app/

 全部内容永久免费开源 · 公众号：AI 产品手艺人 · 小红书：@ 清华学长 harry